

DSA06 - Data Handling and Visualization

List of Experiments – Sets 21 to 30

R Code → Output Visualization → Python Equivalent Code

SET 21: Energy Consumption Data

Dataset: Sector, Region, Month, Temperature, Units_Consumed, Cost, Renewable_Usage, Peak_Hours (6 sample records)

1. Create a histogram with an overlaid density plot for Units_Consumed.
2. Create a scatter plot of Temperature vs Units_Consumed with bubble size representing Peak_Hours.
3. Compute the average Renewable_Usage per Sector and visualize it with a bar chart.
4. Import into Tableau and build an interactive dashboard: line chart of monthly usage trends, a heatmap of Sector vs Region vs Units, with filters for Month/Sector.

R Code

```
energy_df <- data.frame(
  Sector = c("Residential", "Commercial", "Industrial", "Residential", "Commercial", "Industrial"),
  Region = c("North", "South", "West", "East", "North", "South"),
  Month = c("Jan", "Jan", "Feb", "Feb", "Mar", "Mar"),
  Temperature = c(15, 24, 20, 18, 28, 30),
  Units_Consumed = c(320, 540, 880, 350, 610, 920),
  Cost = c(2100, 3600, 5900, 2300, 4100, 6200),
  Renewable_Usage = c(22, 18, 12, 25, 20, 15),
  Peak_Hours = c(4, 6, 8, 5, 7, 9)
)

# 1. Histogram + density plot for Units_Consumed
ggplot(energy_df, aes(x = Units_Consumed)) +
  geom_histogram(aes(y = ..density..), binwidth = 100, fill = "lightblue", color = "black") +
  geom_density(color = "red", size = 1) +
  labs(title = "Units Consumed: Histogram + Density", x = "Units Consumed (kWh)", y = "Density") +
  theme_minimal()

# 2. Scatter: Temperature vs Units_Consumed, bubble size = Peak_Hours
ggplot(energy_df, aes(x = Temperature, y = Units_Consumed, size = Peak_Hours)) +
  geom_point(alpha = 0.5, color = "darkorange") +
  labs(title = "Temperature vs Units Consumed (bubble = Peak Hours)",
       x = "Temperature (C)", y = "Units Consumed (kWh)") +
  theme_minimal()

# 3. Average Renewable_Usage per Sector + bar chart
renewable_avg <- energy_df %>% group_by(Sector) %>% summarise(AvgRenewable = mean(Renewable_Usage))
ggplot(renewable_avg, aes(x = Sector, y = AvgRenewable, fill = Sector)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Renewable Energy Usage by Sector", x = "Sector", y = "Renewable Usage (%)") +
  theme_minimal() + theme(legend.position = "none")

# 4. NOTE: Tableau dashboard (line chart of monthly usage, heatmap of
#       Sector vs Region vs Units, filters) -> done in Tableau, not R.
```

Output Visualization

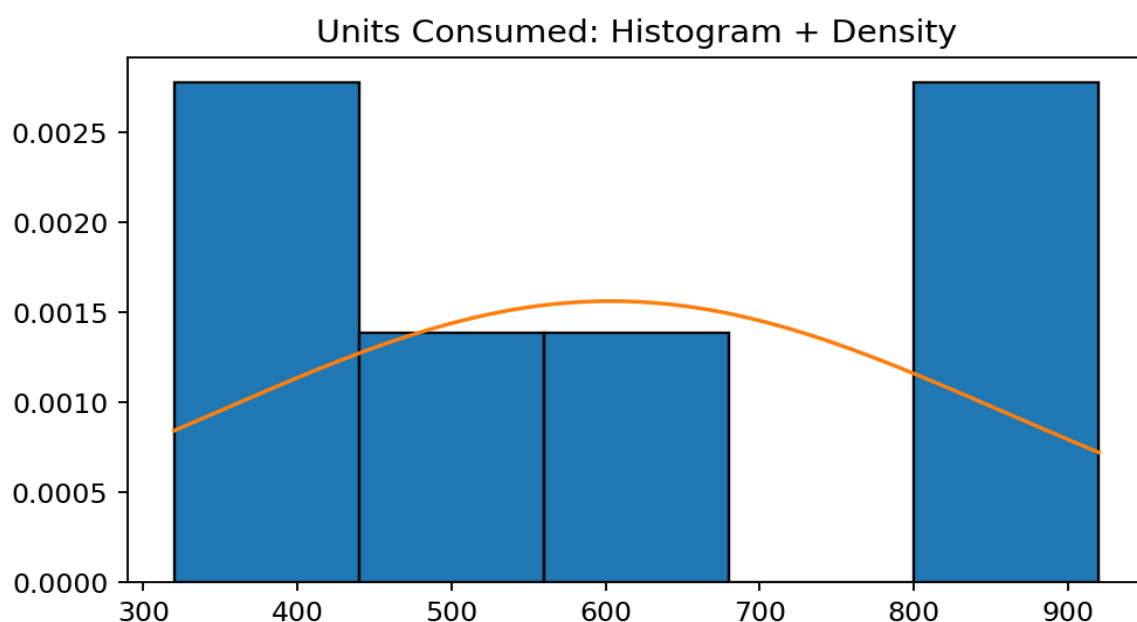


Fig 21.1 – Units Consumed: Histogram + Density

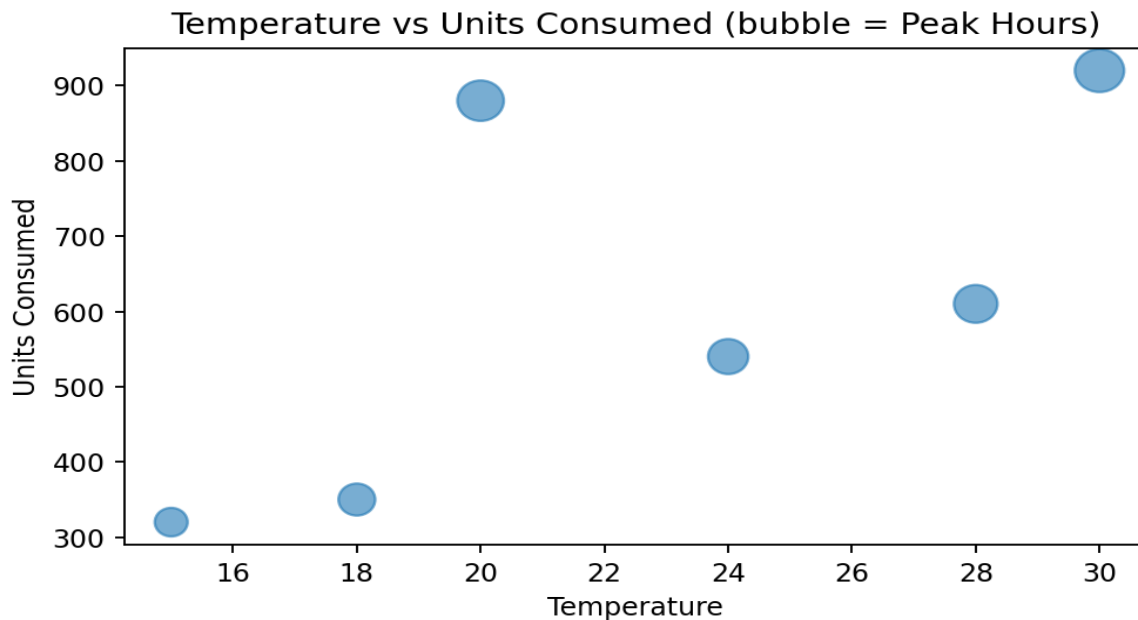


Fig 21.2 – Temperature vs Units Consumed

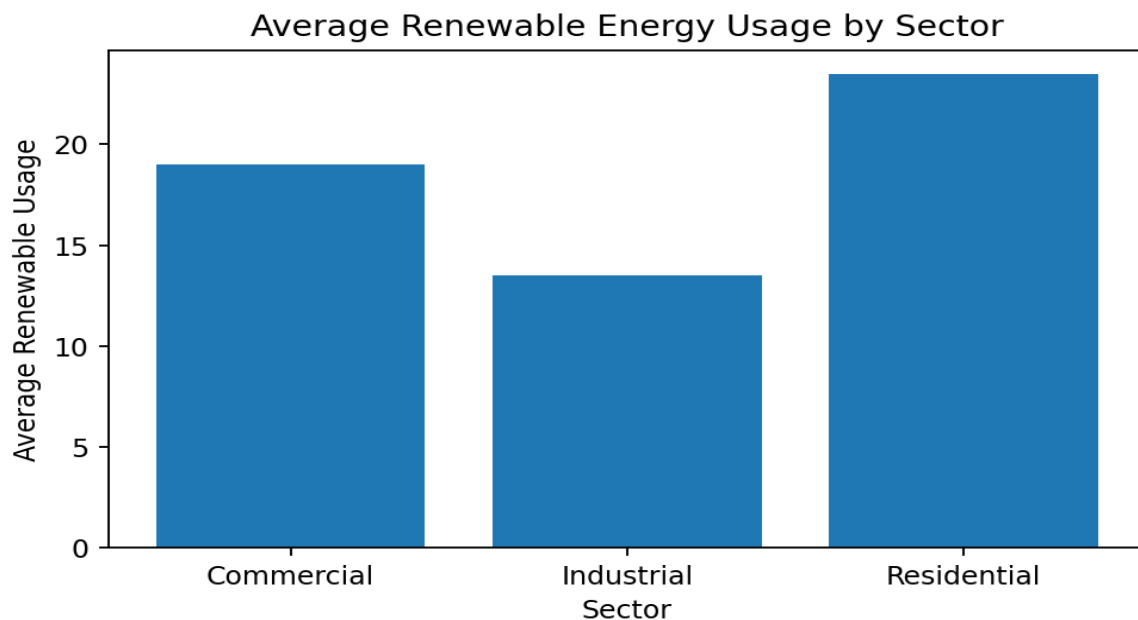


Fig 21.3 – Average Renewable Usage by Sector

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

energy_df = pd.DataFrame({
    "Sector": ["Residential", "Commercial", "Industrial", "Residential", "Commercial", "Industrial"],
    "Region": ["North", "South", "West", "East", "North", "South"],
    "Month": ["Jan", "Jan", "Feb", "Feb", "Mar", "Mar"],
    "Temperature": [15, 24, 20, 18, 28, 30],
    "Units_Consumed": [320, 540, 880, 350, 610, 920],
    "Cost": [2100, 3600, 5900, 2300, 4100, 6200],
    "Renewable_Usage": [22, 18, 12, 25, 20, 15],
    "Peak_Hours": [4, 6, 8, 5, 7, 9]
})

# 1. Histogram + density plot for Units_Consumed
sns.histplot(energy_df["Units_Consumed"], bins=5, stat="density", color="lightblue", edgecolor="black")
sns.kdeplot(energy_df["Units_Consumed"], color="red")
plt.title("Units Consumed: Histogram + Density")
plt.show()

# 2. Scatter: Temperature vs Units_Consumed, bubble size = Peak_Hours
```

```

plt.scatter(energy_df["Temperature"], energy_df["Units_Consumed"],
            s=energy_df["Peak_Hours"] * 30, alpha=0.5, color="darkorange")
plt.title("Temperature vs Units Consumed (bubble = Peak Hours)")
plt.show()

# 3. Average Renewable_Usage per Sector + bar chart
renewable_avg = energy_df.groupby("Sector")["Renewable_Usage"].mean().reset_index()
sns.barplot(data=renewable_avg, x="Sector", y="Renewable_Usage",
            hue="Sector", palette="Greens_d", legend=False)
plt.title("Average Renewable Energy Usage by Sector")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.

```

SET 22: Time Series Analysis

Dataset: Reuses the Monthly Sales data (Month, Sales) and Advertising Budget data (Month, AdBudget, Sales) from Set 1

1. Create a time series line chart of monthly sales.
2. Create a scatter plot of advertising budget vs monthly sales.
3. In Tableau, build a dashboard combining a line chart (sales trend) and a pie chart (sales distribution across products).
4. Create an autocorrelation (ACF) plot to identify any seasonality in the sales data.

R Code

```
# Reuses sales_df and ad_df from Set 1
sales_df <- data.frame(
  Month = factor(c("January", "February", "March", "April", "May"),
    levels = c("January", "February", "March", "April", "May")),
  Sales = c(15000, 18000, 22000, 20000, 23000)
)
ad_df <- data.frame(
  Month = sales_df$Month,
  AdBudget = c(2000, 2500, 3000, 3200),
  Sales = sales_df$Sales
)

# 1. Time series line chart of monthly sales
sales_ts <- ts(sales_df$Sales, start = 1, frequency = 1)
plot(sales_ts, type = "o", col = "blue", main = "Monthly Sales Time Series",
  xlab = "Month (index)", ylab = "Sales ($)")

# 2. Scatter plot: advertising budget vs monthly sales
ggplot(ad_df, aes(x = AdBudget, y = Sales)) +
  geom_point(size = 3, color = "darkred") +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Advertising Budget vs Monthly Sales", x = "Ad Budget ($)", y = "Sales ($)") +
  theme_minimal()

# 3. NOTE: "In Tableau, dashboard combining line chart (sales trend) and
#   pie chart (sales distribution across products)" -> done in Tableau.

# 4. Autocorrelation plot to identify seasonality
acf(sales_df$Sales, main = "Autocorrelation of Monthly Sales")
```

Output Visualization

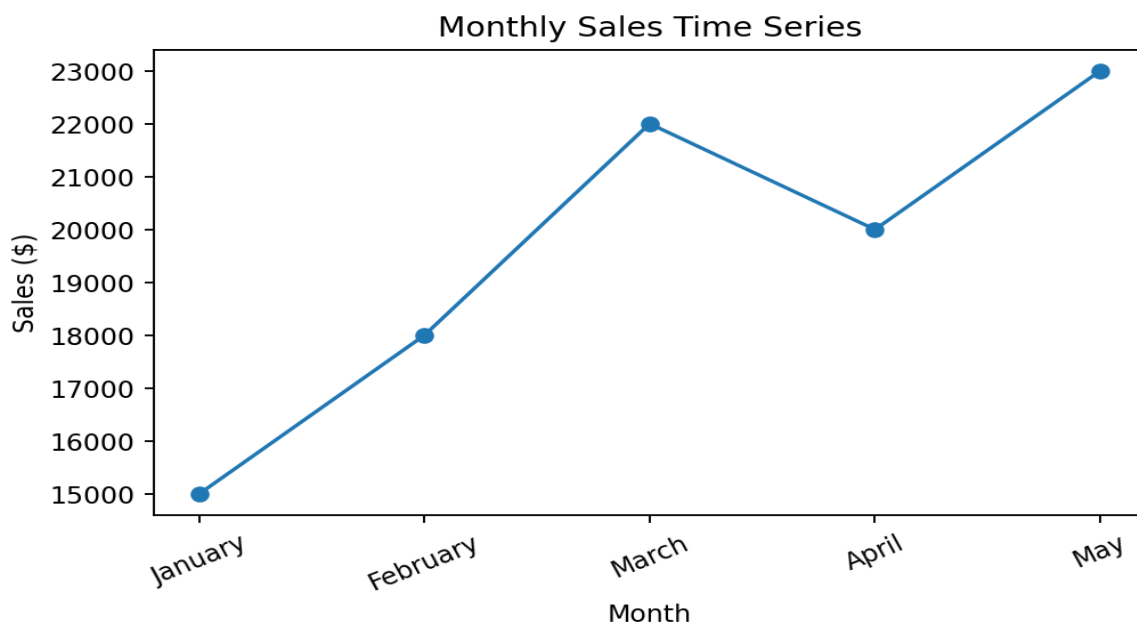


Fig 22.1 – Monthly Sales Time Series

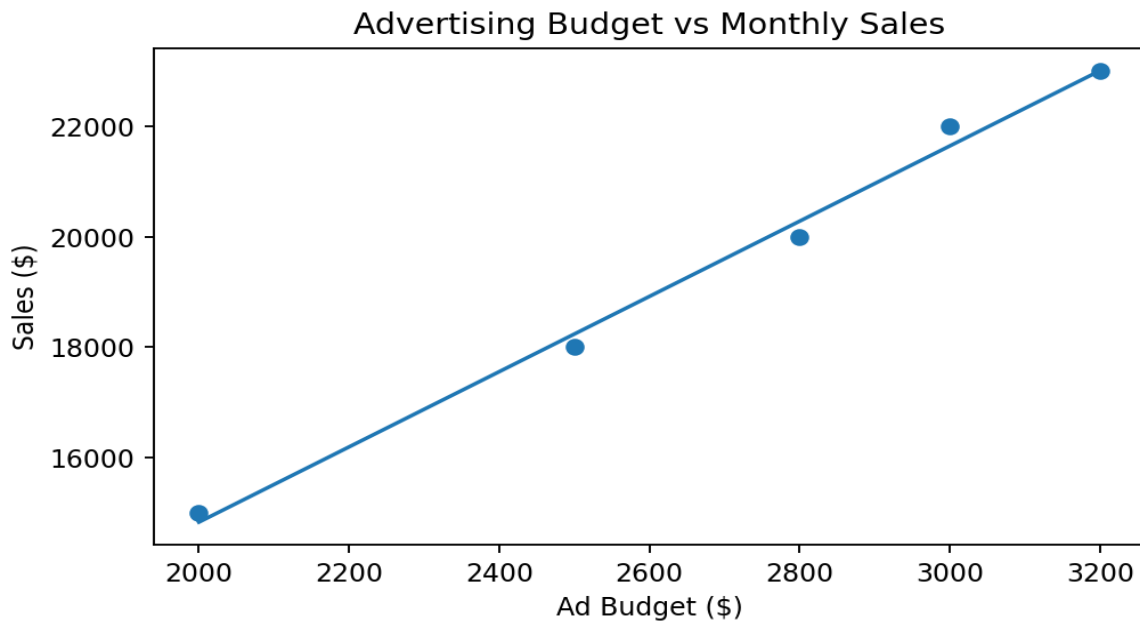


Fig 22.2 – Advertising Budget vs Monthly Sales

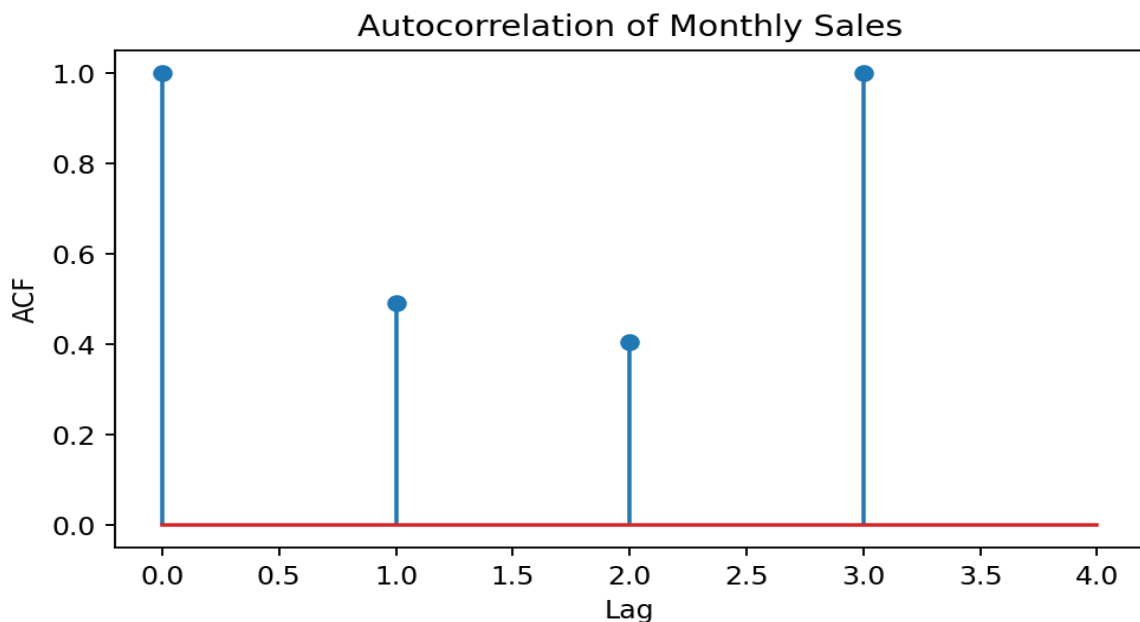


Fig 22.4 – Autocorrelation (ACF) of Monthly Sales

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.graphics.tsaplots import plot_acf

sales_df = pd.DataFrame({
    "Month": ["January", "February", "March", "April", "May"],
    "Sales": [15000, 18000, 22000, 20000, 23000]
})
ad_df = pd.DataFrame({
    "Month": sales_df["Month"],
    "AdBudget": [2000, 2500, 3000, 2800, 3200],
    "Sales": sales_df["Sales"]
})

# 1. Time series line chart of monthly sales
plt.plot(sales_df["Month"], sales_df["Sales"], marker="o", color="blue")
plt.title("Monthly Sales Time Series")
plt.show()

# 2. Scatter plot: advertising budget vs monthly sales
sns.regplot(data=ad_df, x="AdBudget", y="Sales", color="darkred", ci=None)
```

```
plt.title("Advertising Budget vs Monthly Sales")
plt.show()

# 3. Tableau dashboard step done in Tableau, not Python.

# 4. Autocorrelation plot to identify seasonality
fig, ax = plt.subplots()
plot_acf(sales_df["Sales"], ax=ax, lags=4)
plt.title("Autocorrelation of Monthly Sales")
plt.show()
```

SET 23: Employee Performance Evaluation

Dataset: EmployeeID, Department, YearsOfService, PerformanceScore (5 sample employees)

1. Create a line chart to visualize the trend in employee performance.
2. Generate a bar chart showing the distribution of employees across departments.
3. Create a table displaying the employee performance data.
4. Develop an interactive Tableau dashboard combining the line chart and bar chart.

R Code

```
emp_df <- data.frame(
  EmployeeID      = 1:5,
  Department      = c("Sales", "HR", "Marketing", "Sales", "HR"),
  YearsOfService  = c(5, 3, 7, 4, 2),
  PerformanceScore = c(85, 92, 78, 90, 76)
)

# 1. Line chart of performance trend
ggplot(emp_df, aes(x = EmployeeID, y = PerformanceScore)) +
  geom_line(color = "purple") + geom_point(size = 2) +
  labs(title = "Employee Performance Trend", x = "Employee (order)",
       y = "Performance Score") +
  theme_minimal() + scale_x_continuous(breaks = emp_df$EmployeeID)

# 2. Bar chart: employee distribution across departments
dept_counts <- as.data.frame(table(emp_df$Department))
colnames(dept_counts) <- c("Department", "Count")
ggplot(dept_counts, aes(x = Department, y = Count, fill = Department)) +
  geom_bar(stat = "identity") +
  labs(title = "Employees by Department", x = "Department", y = "Count") +
  theme_minimal() + theme(legend.position = "none")

# 3. Table of employee performance data
print(emp_df)

# 4. NOTE: Tableau -> Sheet1 = EmployeeID vs PerformanceScore (line),
#         Sheet2 = Department count (bar) -> combine on one Dashboard.
```

Output Visualization

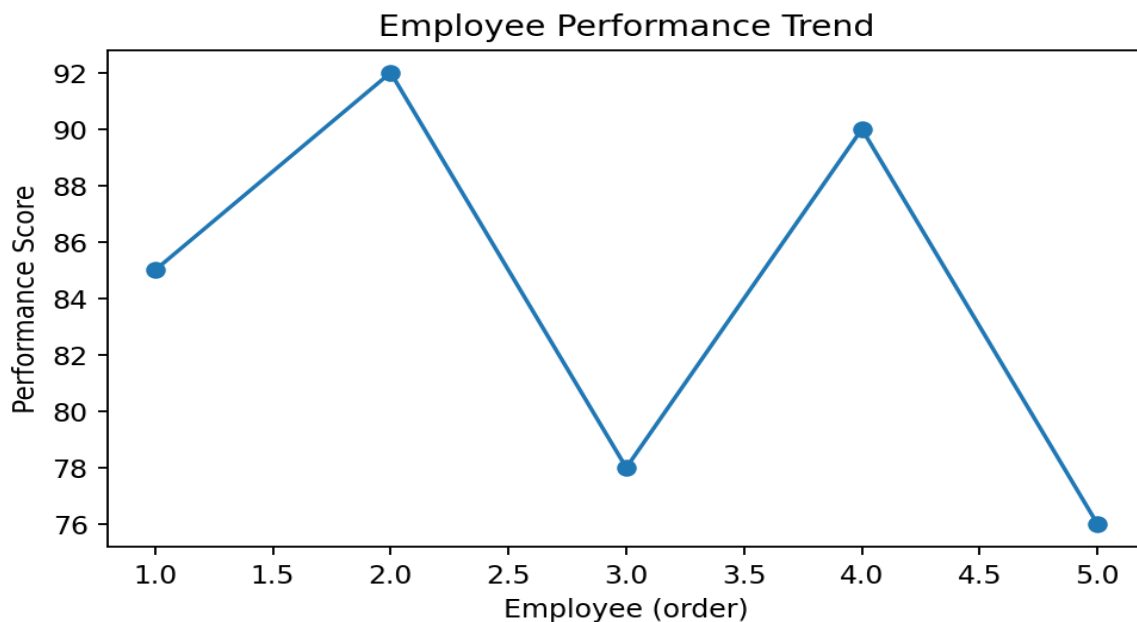


Fig 23.1 – Employee Performance Trend (Line Chart)

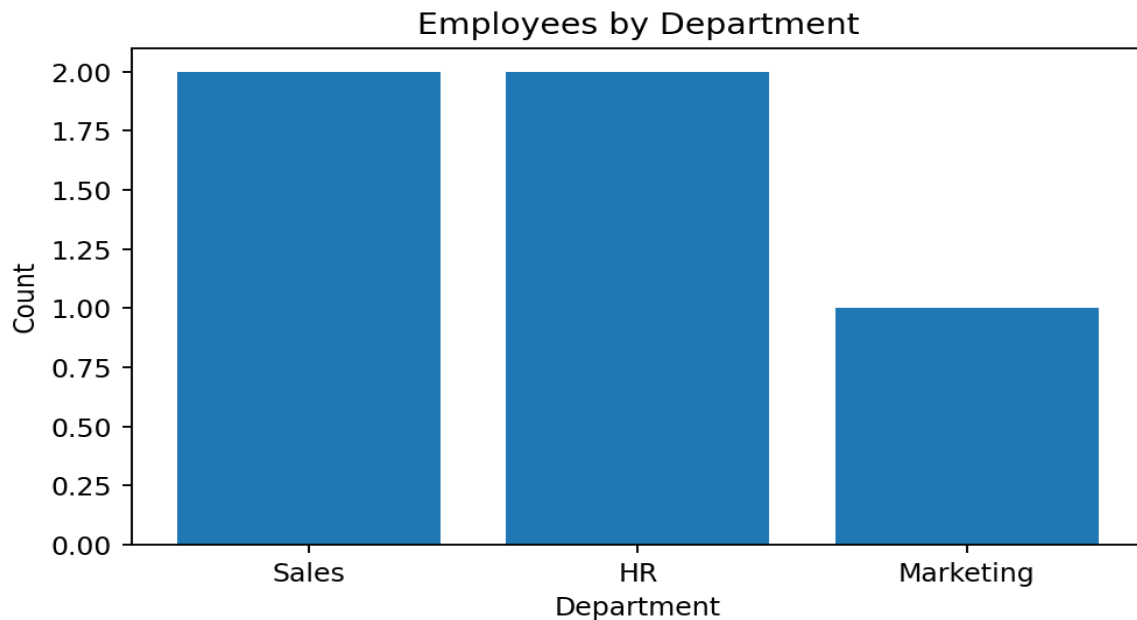


Fig 23.2 – Employees by Department (Bar Chart)

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

emp_df = pd.DataFrame({
    "EmployeeID": [1, 2, 3, 4, 5],
    "Department": ["Sales", "HR", "Marketing", "Sales", "HR"],
    "YearsOfService": [5, 3, 7, 4, 2],
    "PerformanceScore": [85, 92, 78, 90, 76]
})

# 1. Line chart - performance trend
plt.plot(emp_df["EmployeeID"], emp_df["PerformanceScore"], marker="o", color="purple")
plt.title("Employee Performance Trend")
plt.xticks(emp_df["EmployeeID"])
plt.show()

# 2. Bar chart - employees per department
dept_counts = emp_df["Department"].value_counts().reset_index()
dept_counts.columns = ["Department", "Count"]
sns.barplot(data=dept_counts, x="Department", y="Count",
            hue="Department", palette="Set2", legend=False)
plt.title("Employees by Department")
plt.show()

# 3. Table of employee performance data
print(emp_df)

# 4. Tableau dashboard step done in Tableau, not Python.
```

SET 24: Product Inventory Management

Dataset: ProductID, ProductName, Quantity, Price (5 sample products; Category assumed for grouping)

1. Create a bar chart to visualize the quantity of each product available.
2. Generate a stacked bar chart to show the quantity of products by category.
3. Create a table displaying the inventory data.
4. Develop an interactive Tableau dashboard combining the bar chart and stacked bar chart.

R Code

```
inv_df <- data.frame(  
  ProductID = 1:5,  
  ProductName = c("Product A", "Product B", "Product C", "Product D", "Product E"),  
  Quantity = c(250, 175, 300, 200, 220),  
  Price = c(20, 15, 18, 22, 19) # ASSUMED for Set 24  
)  
inv_df$Category <- c("Electronics", "Electronics", "Home", "Home", "Office") # ASSUMED  
  
# 1. Bar chart of quantity available per product  
ggplot(inv_df, aes(x = ProductName, y = Quantity, fill = ProductName)) +  
  geom_bar(stat = "identity") +  
  labs(title = "Product Inventory Quantities", x = "Product", y = "Quantity Available") +  
  theme_minimal() + theme(legend.position = "none")  
  
# 2. Stacked bar chart: quantity by product category  
ggplot(inv_df, aes(x = Category, y = Quantity, fill = ProductName)) +  
  geom_bar(stat = "identity", position = "stack") +  
  labs(title = "Quantity by Product Category", x = "Category", y = "Quantity") +  
  theme_minimal()  
  
# 3. Table of inventory data  
print(inv_df)  
  
# 4. NOTE: Tableau -> Sheet1 = ProductName vs Quantity (bar), Sheet2 =  
#   Category vs Quantity coloured by ProductName (stacked bar) -> Dashboard.
```

Output Visualization

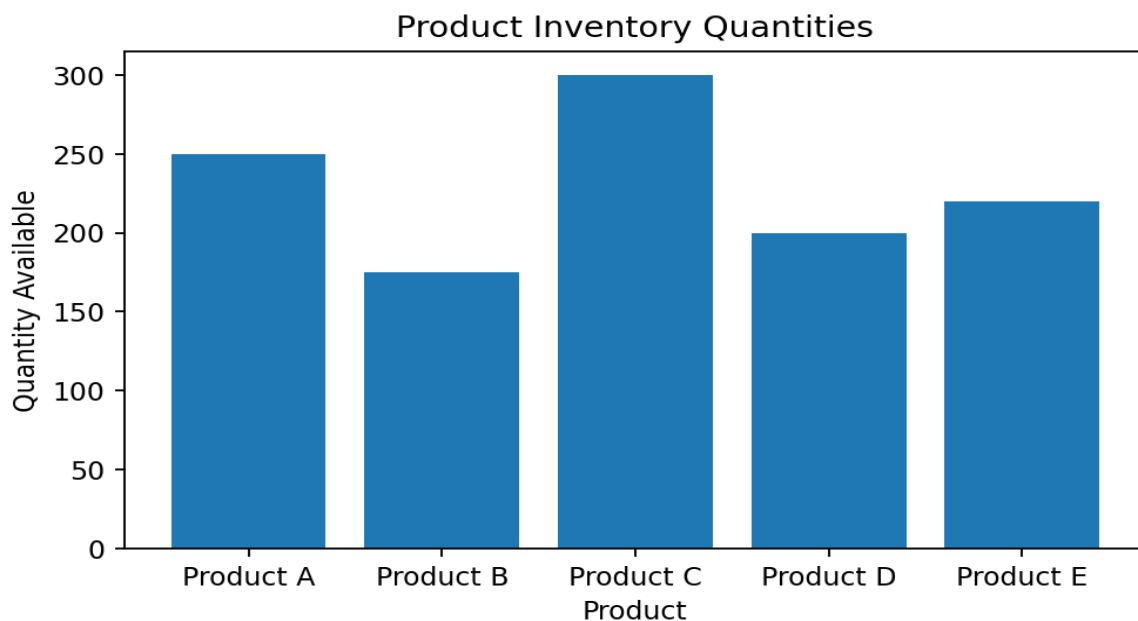


Fig 24.1 – Product Inventory Quantities (Bar Chart)

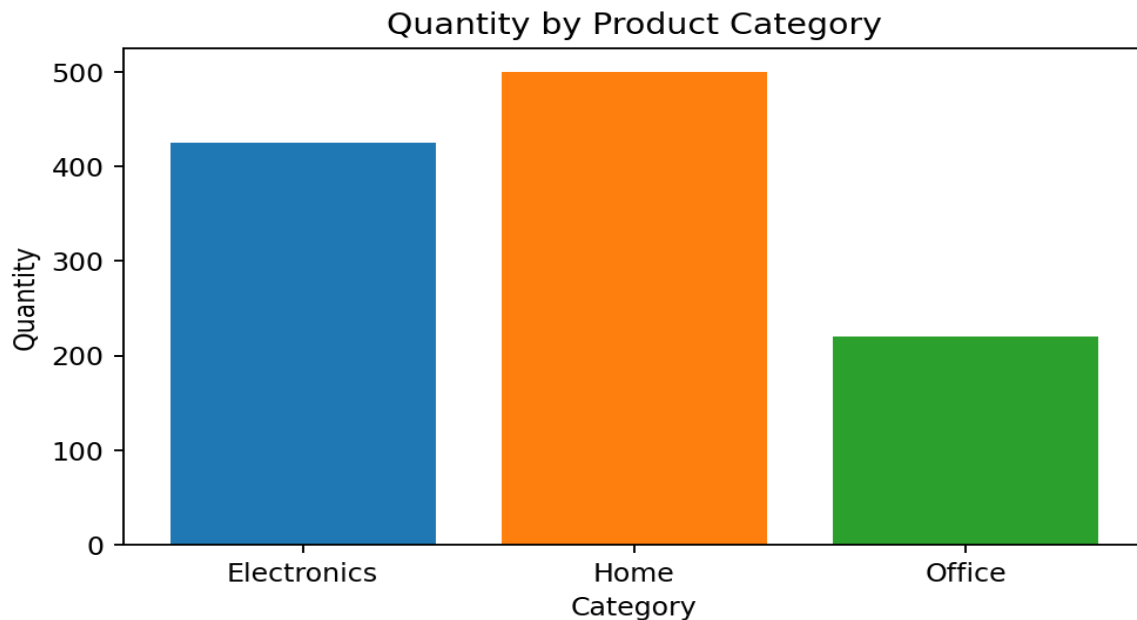


Fig 24.2 – Quantity by Product Category (Stacked Bar Chart)

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

inv_df = pd.DataFrame({
    "ProductID": [1, 2, 3, 4, 5],
    "ProductName": ["Product A", "Product B", "Product C", "Product D", "Product E"],
    "Quantity": [250, 175, 300, 200, 220],
    "Price": [20, 15, 18, 22, 19],
    "Category": ["Electronics", "Electronics", "Home", "Home", "Office"] # ASSUMED
})

# 1. Bar chart - quantity per product
sns.barplot(data=inv_df, x="ProductName", y="Quantity",
            hue="ProductID", palette="mako", legend=False)
plt.title("Product Inventory Quantities")
plt.show()

# 2. Stacked bar chart - quantity by category
pivot = inv_df.pivot_table(index="Category", columns="ProductID",
                            values="Quantity", aggfunc="sum", fill_value=0)
pivot.plot(kind="bar", stacked=True, colormap="tab10")
plt.title("Quantity by Product Category")
plt.show()

# 3. Table of inventory data
print(inv_df)

# 4. Tableau dashboard step done in Tableau, not Python.
```

SET 25: Website Analytics / Traffic

Dataset: Date, PageViews, CTR (5 days); Source, Lat, Lon assumed for the traffic-source map task

1. Create a line chart to visualize daily page views over time.
2. Create an interactive map showing traffic sources, with bubble size representing page views.
3. Create a bar chart of page views by traffic source.
4. Develop an interactive Tableau dashboard combining the map and the bar chart, using Tableau's native geographic mapping.

R Code

```
web_df <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05")),
  PageViews = c(1500, 1600, 1400, 1650, 1800),
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6),
  Source = c("Direct", "Search", "Social", "Referral", "Direct"), # ASSUMED
  Lat = c(28.6139, 19.0760, 13.0827, 22.5726, 12.9716), # ASSUMED
  Lon = c(77.2090, 72.8777, 80.2707, 88.3639, 77.5946) # ASSUMED
)

# 1. Line chart: daily page views over time
ggplot(web_df, aes(x = Date, y = PageViews)) +
  geom_line(color = "tomato") + geom_point() +
  labs(title = "Daily Page Views Trend", x = "Date", y = "Page Views") +
  theme_minimal()

# 2. Interactive map of traffic sources (bubble size = PageViews)
# plotly / Tableau give a true interactive map; a static bubble-by-location
# view is shown below for the report (color = Source, size = PageViews)
ggplot(web_df, aes(x = Lon, y = Lat, size = PageViews, color = Source)) +
  geom_point(alpha = 0.7) +
  labs(title = "Traffic Source Locations (bubble = Page Views)", x = "Longitude", y = "Latitude") +
  theme_minimal()

# 3. Bar chart: page views by traffic source
ggplot(web_df, aes(x = Source, y = PageViews, fill = Source)) +
  geom_bar(stat = "identity") +
  labs(title = "Page Views by Traffic Source", x = "Source", y = "Page Views") +
  theme_minimal() + theme(legend.position = "none")

# 4. NOTE: Tableau dashboard (interactive map + bar chart, native geo
# mapping) -> done in Tableau, not R.
```

Output Visualization

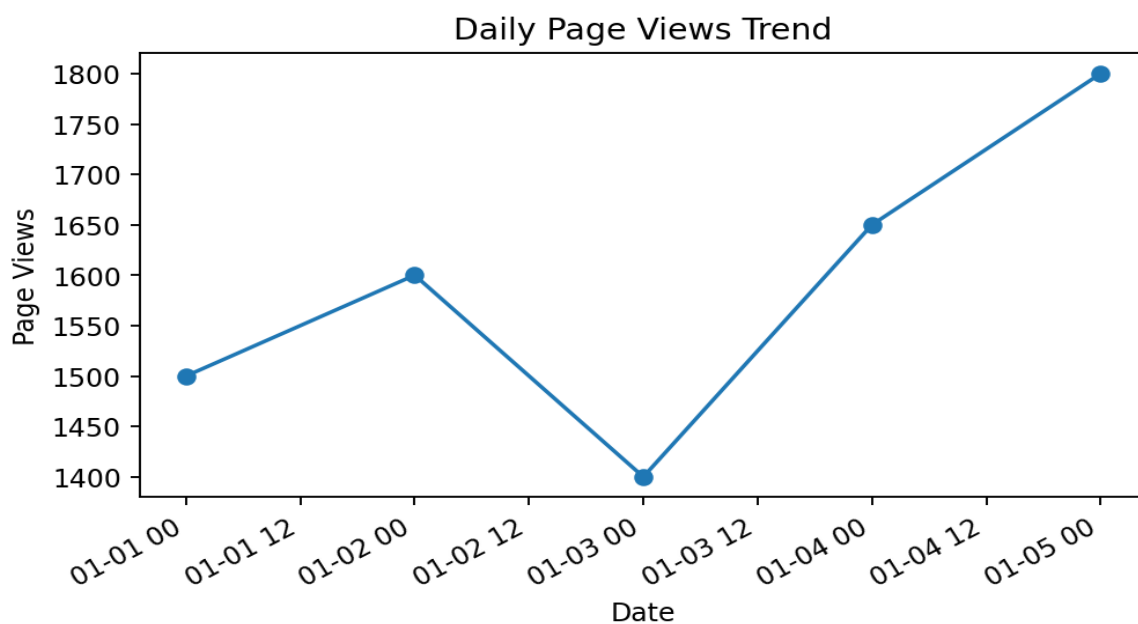


Fig 25.1 – Daily Page Views Trend (Line Chart)

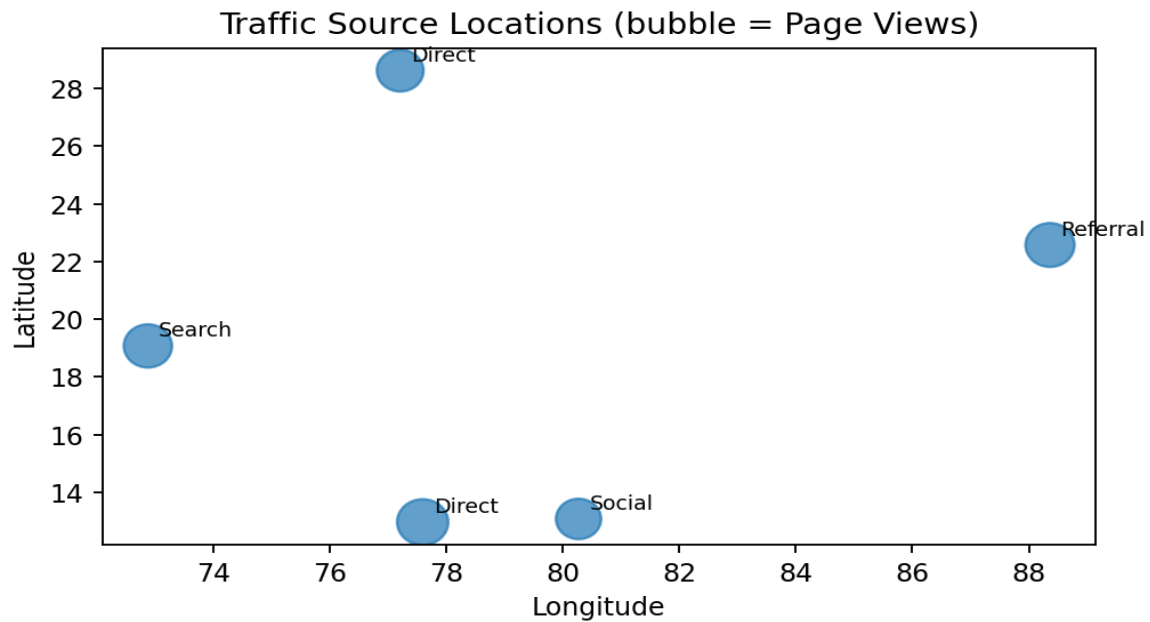


Fig 25.2 – Traffic Source Locations (Bubble = Page Views)

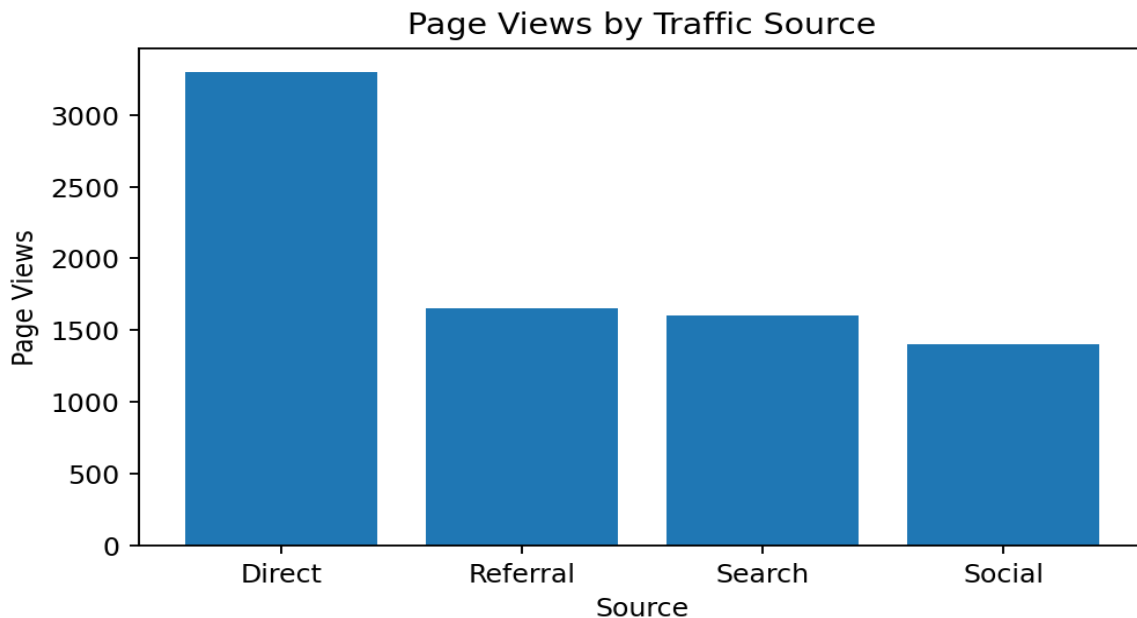


Fig 25.3 – Page Views by Traffic Source (Bar Chart)

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

web_df = pd.DataFrame({
    "Date": pd.to_datetime(["2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05"]),
    "PageViews": [1500, 1600, 1400, 1650, 1800],
    "CTR": [2.3, 2.7, 2.0, 2.4, 2.6],
    "Source": ["Direct", "Search", "Social", "Referral", "Direct"], # ASSUMED
    "Lat": [28.6139, 19.0760, 13.0827, 22.5726, 12.9716],
    "Lon": [77.2090, 72.8777, 80.2707, 88.3639, 77.5946]
})

# 1. Line chart: daily page views over time
plt.plot(web_df["Date"], web_df["PageViews"], marker="o", color="tomato")
plt.title("Daily Page Views Trend")
plt.show()

# 2. Bubble map of traffic source locations (bubble size = PageViews)
for src in web_df["Source"].unique():
    sub = web_df[web_df["Source"] == src]
    plt.scatter(sub["Lon"], sub["Lat"], s=sub["PageViews"] / 5, label=src, alpha=0.7)
```

```
plt.title("Traffic Source Locations (bubble = Page Views)")
plt.legend()
plt.show()

# 3. Bar chart: page views by traffic source
sns.barplot(data=web_df, x="Source", y="PageViews",
            hue="Source", palette="viridis", legend=False)
plt.title("Page Views by Traffic Source")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.
```

SET 26: Student Mini Data

Dataset: Student_ID, Gender, Study_Hours, Attendance, Math_Score, Science_Score, Exam_Date (6 students)

1. Create a histogram of Math_Score and a boxplot of Science_Score grouped by Gender.
2. Create a scatter plot of Study_Hours vs Math_Score, coloured by Gender, with a regression line.
3. Compute the monthly average Math score and plot a line chart with a moving average.
4. Import into Tableau: bar chart of average Math scores by Gender, a line chart for score trends over time, and a filter for Exam_Date, in an interactive dashboard.

R Code

```
student_df <- data.frame(
  Student_ID = c("S01", "S02", "S03", "S04", "S05", "S06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Study_Hours = c(2.0, 3.5, 1.5, 4.0, 2.8, 3.0),
  Attendance = c(78, 90, 70, 95, 85, 92),
  Math_Score = c(62, 80, 55, 90, 72, 82),
  Science_Score = c(65, 85, 58, 92, 74, 86),
  Exam_Date = as.Date(c("2025-01-10", "2025-01-10", "2025-02-12",
                        "2025-02-12", "2025-03-15", "2025-03-15"))
)

# 1. Histogram of Math_Score + boxplot of Science_Score by Gender
ggplot(student_df, aes(x = Math_Score)) +
  geom_histogram(binwidth = 5, fill = "lightgreen", color = "black") +
  labs(title = "Distribution of Math Scores", x = "Math Score", y = "Frequency") +
  theme_minimal()

ggplot(student_df, aes(x = Gender, y = Science_Score, fill = Gender)) +
  geom_boxplot() +
  labs(title = "Science Score by Gender", x = "Gender", y = "Science Score") +
  theme_minimal()

# 2. Scatter: Study_Hours vs Math_Score, colored by Gender, with regression line
ggplot(student_df, aes(x = Study_Hours, y = Math_Score, color = Gender)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Study Hours vs Math Score by Gender", x = "Study Hours", y = "Math Score") +
  theme_minimal()

# 3. Monthly average Math score, line chart with moving average
student_df$Exam_Month <- format(student_df$Exam_Date, "%Y-%m")
monthly_math <- student_df %>%
  group_by(Exam_Month) %>%
  summarise(AvgMath = mean(Math_Score)) %>%
  arrange(Exam_Month)
monthly_math$MovingAvg <- rollmean(monthly_math$AvgMath, k = 2, fill = NA, align = "right")

ggplot(monthly_math, aes(x = Exam_Month, group = 1)) +
  geom_line(aes(y = AvgMath), color = "steelblue", size = 1) +
  geom_line(aes(y = MovingAvg), color = "red", linetype = "dashed", size = 1) +
  labs(title = "Monthly Average Math Score with Moving Average", x = "Month", y = "Average Math Score") +
  theme_minimal()

# 4. NOTE: Tableau dashboard (bar chart of avg Math by Gender, line chart
# of score trends, Exam_Date filter) -> done in Tableau, not R.
```

Output Visualization

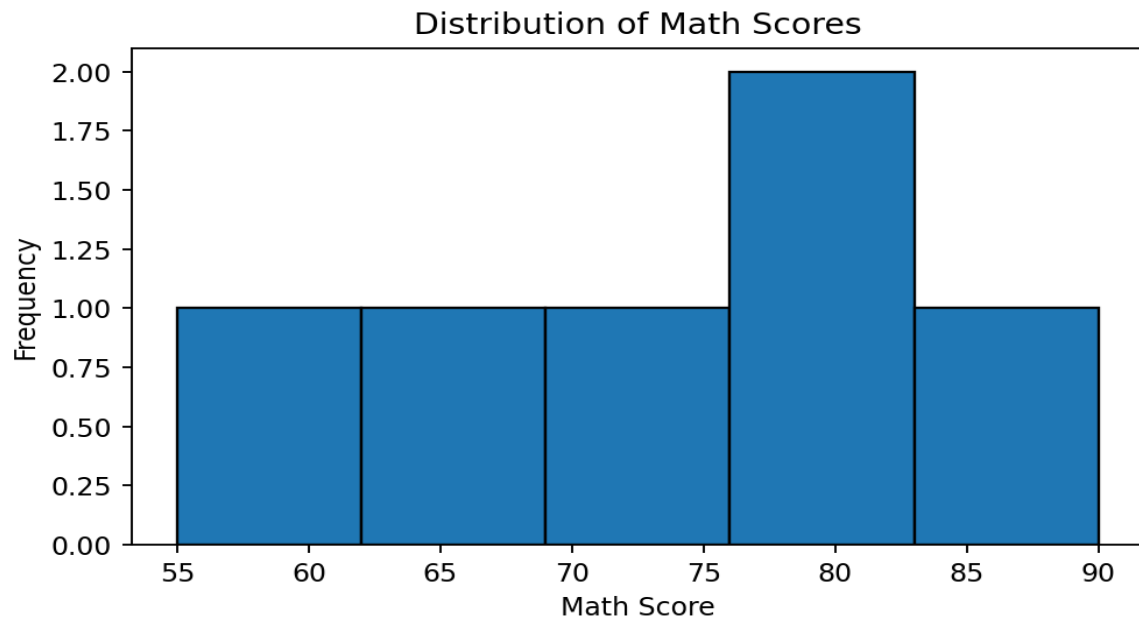


Fig 26.1a – Distribution of Math Scores (Histogram)

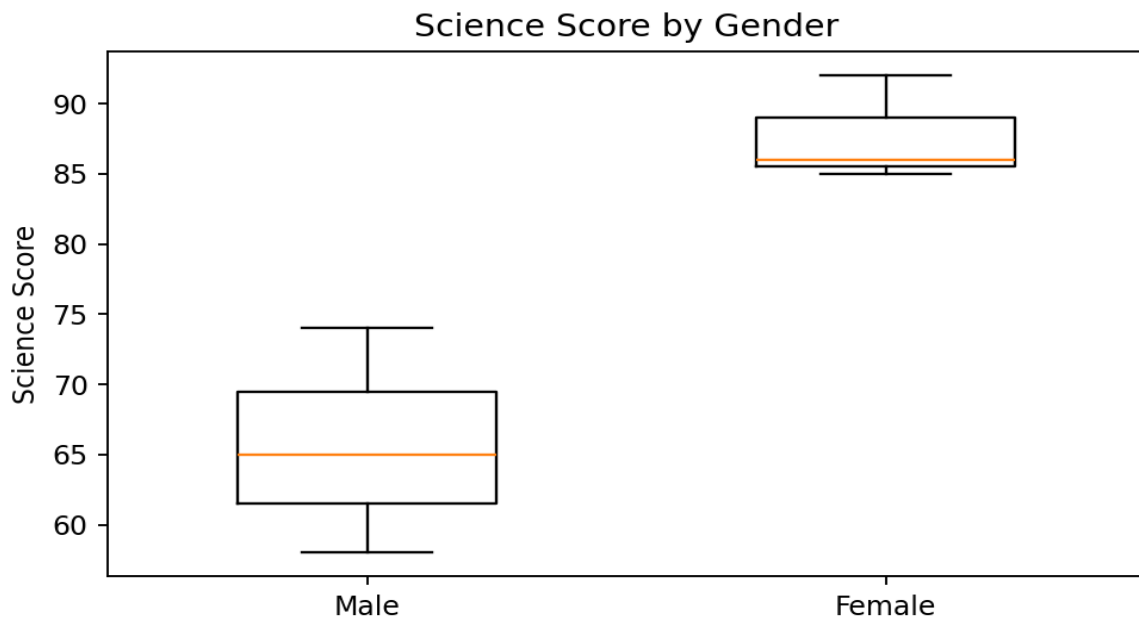


Fig 26.1b – Science Score by Gender (Boxplot)

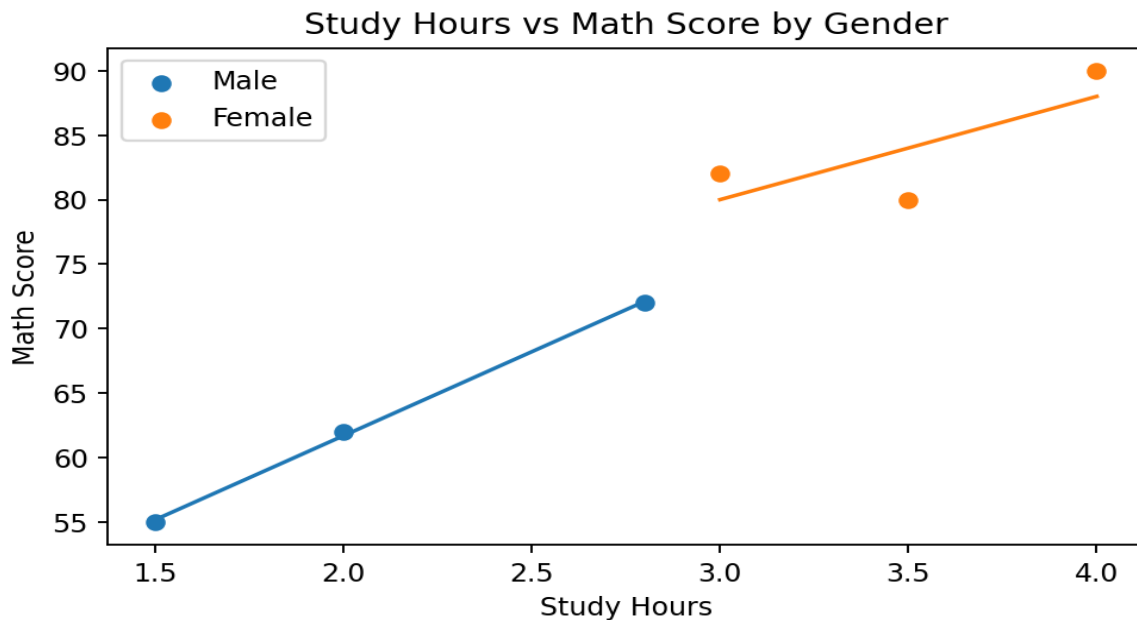


Fig 26.2 – Study Hours vs Math Score by Gender

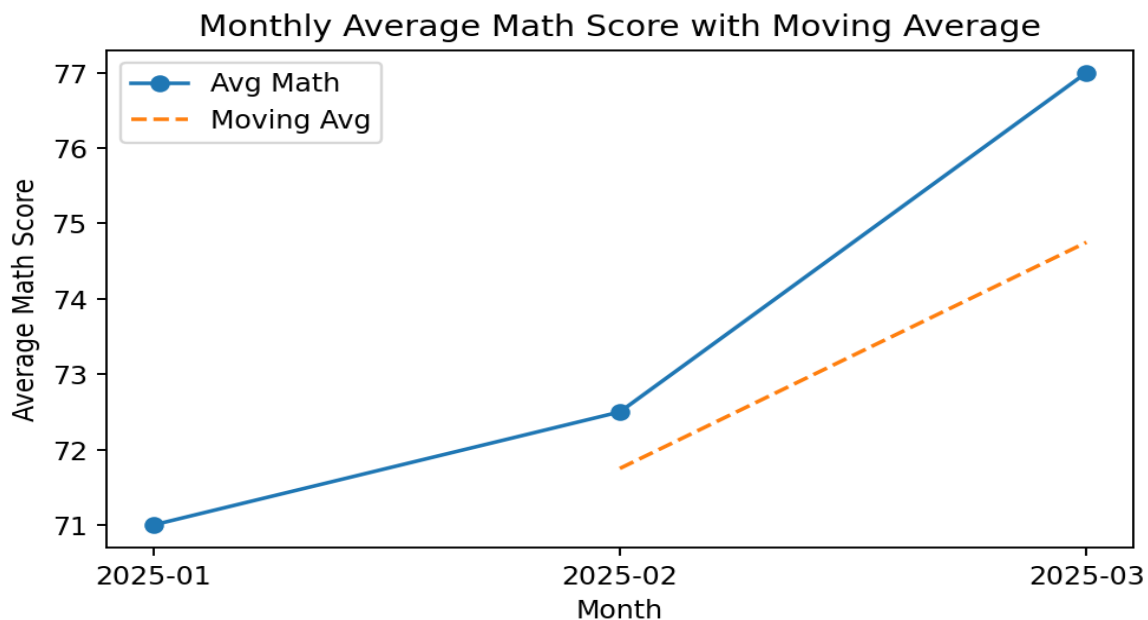


Fig 26.3 – Monthly Average Math Score with Moving Average

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

student_df = pd.DataFrame({
    "Student_ID": ["S01", "S02", "S03", "S04", "S05", "S06"],
    "Gender": ["Male", "Female", "Male", "Female", "Male", "Female"],
    "Study_Hours": [2.0, 3.5, 1.5, 4.0, 2.8, 3.0],
    "Attendance": [78, 90, 70, 95, 85, 92],
    "Math_Score": [62, 80, 55, 90, 72, 82],
    "Science_Score": [65, 85, 58, 92, 74, 86],
    "Exam_Date": pd.to_datetime(["2025-01-10", "2025-01-10", "2025-02-12",
                                "2025-02-12", "2025-03-15", "2025-03-15"])
})

# 1. Histogram of Math_Score + boxplot of Science_Score by Gender
sns.histplot(student_df["Math_Score"], bins=5, color="lightgreen", edgecolor="black")
plt.title("Distribution of Math Scores")
plt.show()

sns.boxplot(data=student_df, x="Gender", y="Science_Score", hue="Gender", legend=False)
plt.title("Science Score by Gender")
```

```

plt.show()

# 2. Scatter: Study_Hours vs Math_Score, colored by Gender, with regression line
sns.lmplot(data=student_df, x="Study_Hours", y="Math_Score", hue="Gender", ci=None)
plt.title("Study Hours vs Math Score by Gender")
plt.show()

# 3. Monthly average Math score, line chart with moving average
student_df["Exam_Month"] = student_df["Exam_Date"].dt.strftime("%Y-%m")
monthly_math = student_df.groupby("Exam_Month")["Math_Score"].mean().reset_index()
monthly_math.columns = ["Exam_Month", "AvgMath"]
monthly_math["MovingAvg"] = monthly_math["AvgMath"].rolling(2).mean()

plt.plot(monthly_math["Exam_Month"], monthly_math["AvgMath"], marker="o", label="Avg Math")
plt.plot(monthly_math["Exam_Month"], monthly_math["MovingAvg"], linestyle="--", color="red", label="Moving Avg")
plt.title("Monthly Average Math Score with Moving Average")
plt.legend()
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.

```

SET 27: Patient Health Risk Analysis

Dataset: Patient_ID, Age, BMI, BP, Cholesterol (5 sample patients)

1. Create a scatterplot matrix for Age, BMI, BP, and Cholesterol to examine pairwise relationships.
2. Create a Q-Q plot and an ECDF plot for Cholesterol to check the distribution's normality.
3. Compute the average of each health indicator and visualize it with a bar chart.
4. Import into Tableau and build an interactive dashboard: a heatmap of high-risk patients, with an Age filter and hover tooltips.

R Code

```
patient_df <- data.frame(
  Patient_ID = c("P1", "P2", "P3", "P4", "P5"),
  Age = c(25, 40, 55, 35, 60),
  BMI = c(22, 28, 30, 26, 32),
  BP = c(120, 135, 145, 130, 150),
  Cholesterol = c(180, 210, 240, 200, 260)
)

# 1. Scatterplot matrix for Age, BMI, BP, Cholesterol
ggpairs(patient_df[, c("Age", "BMI", "BP", "Cholesterol")],
  title = "Scatterplot Matrix - Patient Health Indicators")

# 2. Q-Q plot and ECDF for Cholesterol
qqnorm(patient_df$Cholesterol, main = "Q-Q Plot of Cholesterol")
qqline(patient_df$Cholesterol, col = "red")

ggplot(patient_df, aes(x = Cholesterol)) +
  stat_ecdf(geom = "step", color = "blue", size = 1) +
  labs(title = "ECDF of Cholesterol Levels", x = "Cholesterol", y = "Cumulative Probability") +
  theme_minimal()

# 3. Bar chart of average values of each health indicator
health_avg <- data.frame(
  Indicator = c("Age", "BMI", "BP", "Cholesterol"),
  Average = c(mean(patient_df$Age), mean(patient_df$BMI),
    mean(patient_df$BP), mean(patient_df$Cholesterol))
)
ggplot(health_avg, aes(x = Indicator, y = Average, fill = Indicator)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Patient Health Indicators", x = "Indicator", y = "Average Value") +
  theme_minimal()

# 4. NOTE: Tableau dashboard (heatmap of high-risk patients, Age filter,
#   hover tooltips) -> done in Tableau, not R.
```

Output Visualization

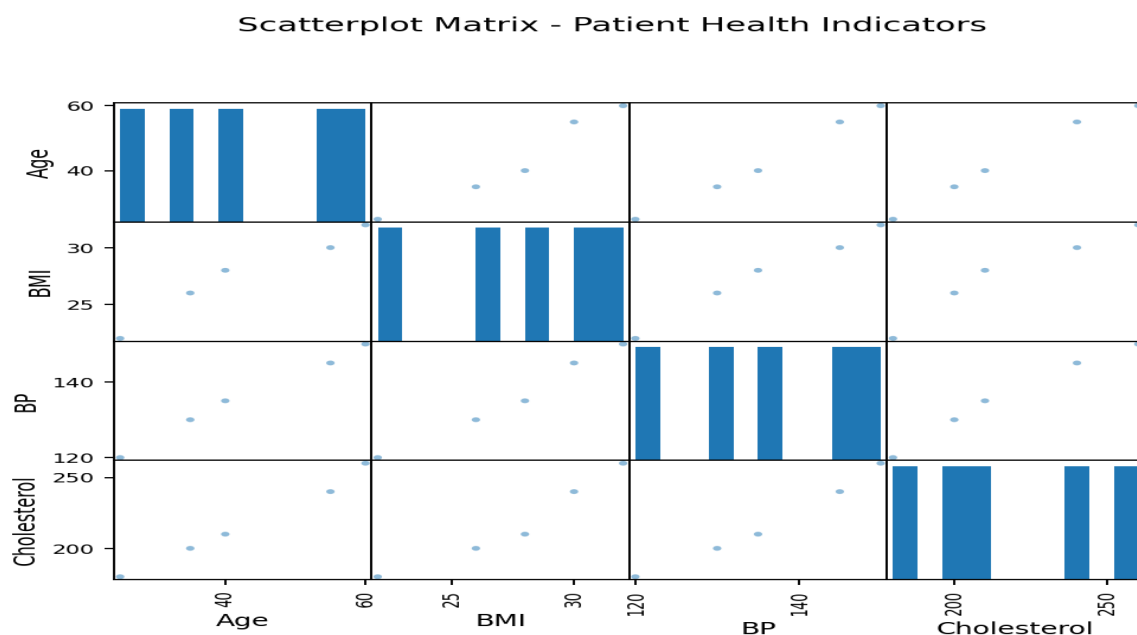


Fig 27.1 – Scatterplot Matrix of Patient Health Indicators

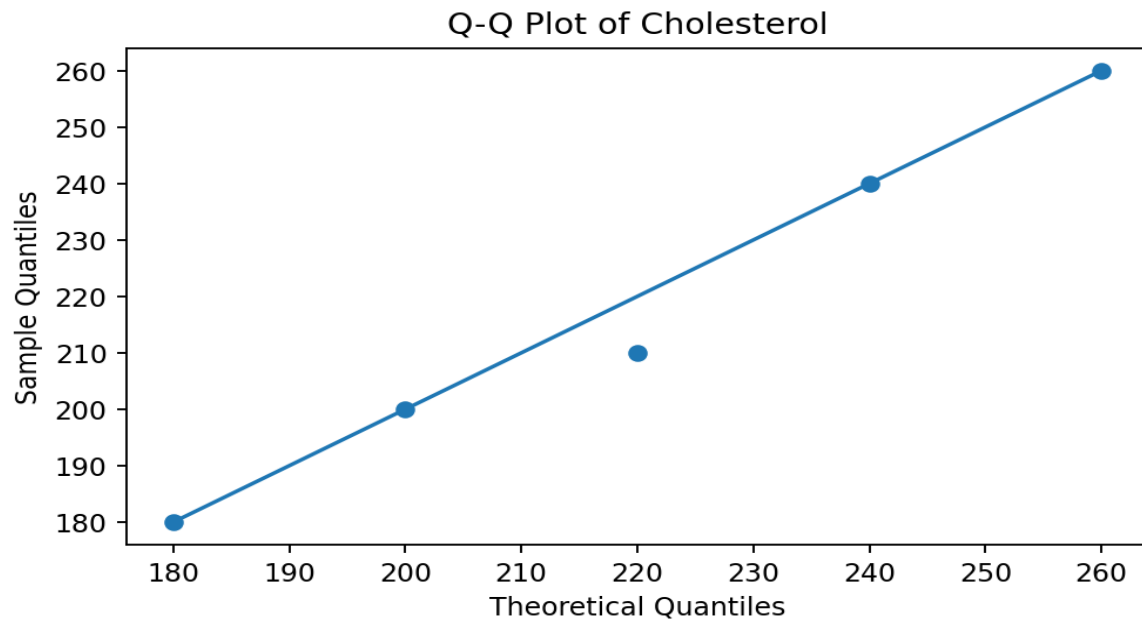


Fig 27.2a – Q-Q Plot of Cholesterol

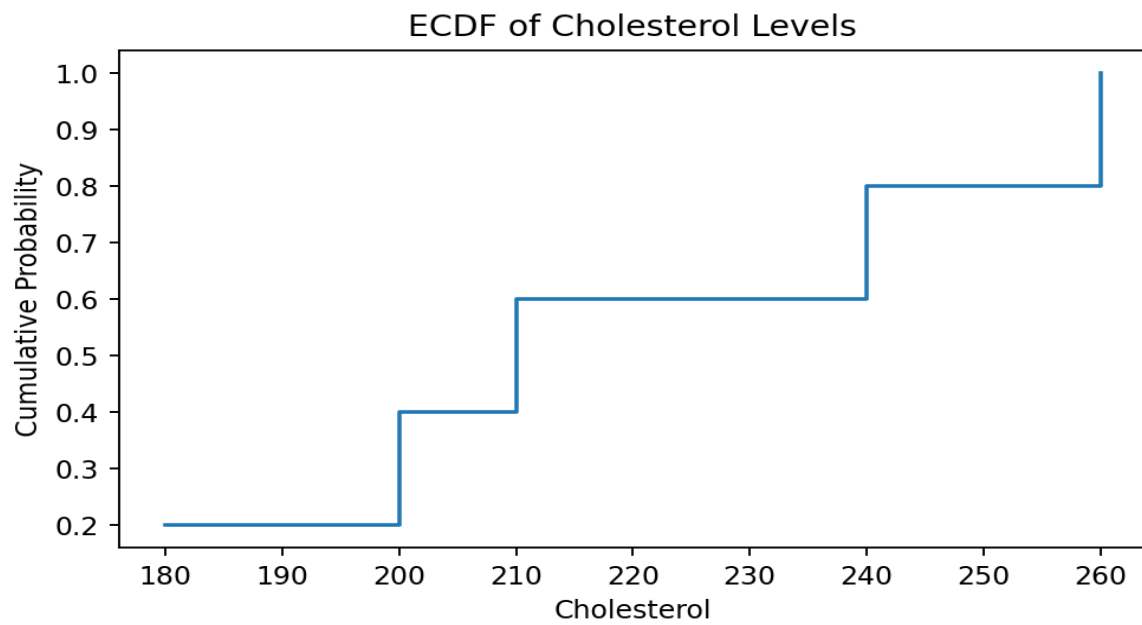


Fig 27.2b – ECDF of Cholesterol Levels

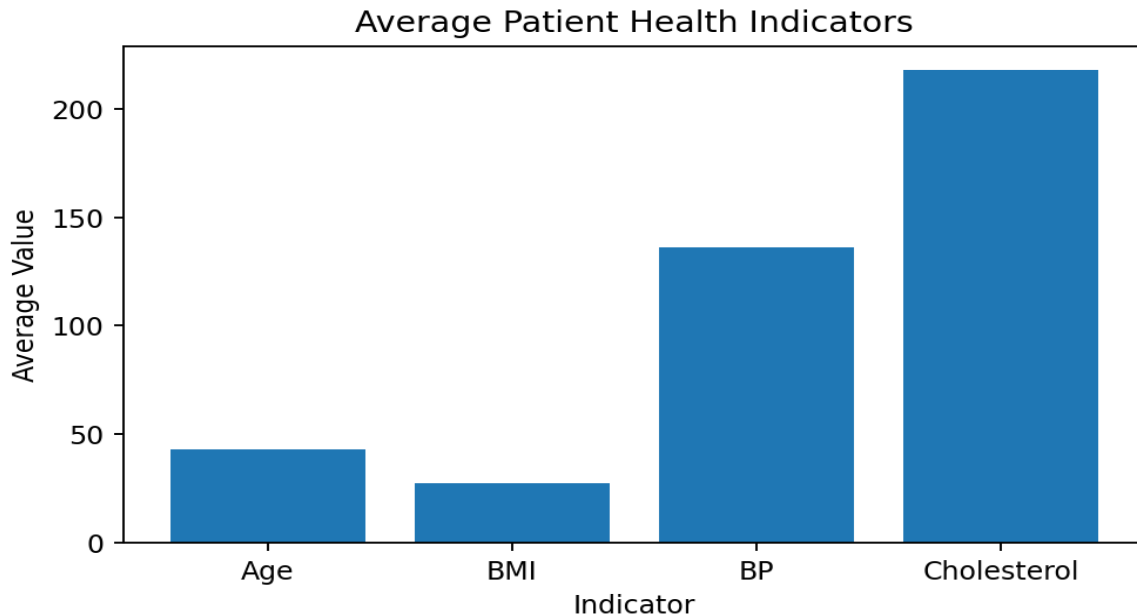


Fig 27.3 – Average Patient Health Indicators

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

patient_df = pd.DataFrame({
    "Patient_ID": ["P1", "P2", "P3", "P4", "P5"],
    "Age": [25, 40, 55, 35, 60],
    "BMI": [22, 28, 30, 26, 32],
    "BP": [120, 135, 145, 130, 150],
    "Cholesterol": [180, 210, 240, 200, 260]
})

# 1. Scatterplot matrix for Age, BMI, BP, Cholesterol
sns.pairplot(patient_df[["Age", "BMI", "BP", "Cholesterol"]])
plt.suptitle("Scatterplot Matrix - Patient Health Indicators", y=1.02)
plt.show()

# 2. Q-Q plot and ECDF for Cholesterol
stats.probplot(patient_df["Cholesterol"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Cholesterol")
plt.show()

sns.ecdfplot(patient_df["Cholesterol"], color="blue")
plt.title("ECDF of Cholesterol Levels")
plt.show()

# 3. Bar chart of average values of each health indicator
health_avg = pd.DataFrame({
    "Indicator": ["Age", "BMI", "BP", "Cholesterol"],
    "Average": [patient_df["Age"].mean(), patient_df["BMI"].mean(),
               patient_df["BP"].mean(), patient_df["Cholesterol"].mean()]
})
sns.barplot(data=health_avg, x="Indicator", y="Average",
            hue="Indicator", palette="coolwarm", legend=False)
plt.title("Average Patient Health Indicators")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.
```

SET 28: Vehicle Performance Analysis

Dataset: Vehicle_ID, Engine_Size, Horsepower, Fuel_Efficiency, Top_Speed, Safety_Rating (5 sample vehicles)

1. Create a violin plot of Fuel_Efficiency across Safety_Rating categories.
2. Create a scatter plot of Horsepower vs Top_Speed, coloured by Engine_Size.
3. Compute the correlation matrix of all numeric variables and visualize it as a heatmap.
4. Import into Tableau and build an interactive dashboard with bar charts and scatter plots, using filters for Engine_Size and Safety_Rating.

R Code

```
vehicle_df <- data.frame(
  Vehicle_ID = c("V1", "V2", "V3", "V4", "V5"),
  Engine_Size = c(1.5, 2.0, 3.0, 2.5, 1.8),
  Horsepower = c(110, 150, 250, 200, 130),
  Fuel_Efficiency = c(18, 15, 12, 14, 17),
  Top_Speed = c(180, 200, 250, 220, 190),
  Safety_Rating = c(4, 5, 5, 4, 3)
)

# 1. Violin plot: Fuel_Efficiency across Safety_Rating categories
ggplot(vehicle_df, aes(x = factor(Safety_Rating), y = Fuel_Efficiency, fill = factor(Safety_Rating))) +
  geom_violin() +
  labs(title = "Fuel Efficiency by Safety Rating", x = "Safety Rating", y = "Fuel Efficiency (km/l)") +
  theme_minimal() + theme(legend.position = "none")

# 2. Scatter: Horsepower vs Top_Speed, color by Engine_Size
ggplot(vehicle_df, aes(x = Horsepower, y = Top_Speed, color = Engine_Size)) +
  geom_point(size = 4) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(title = "Horsepower vs Top Speed (color = Engine Size)", x = "Horsepower", y = "Top Speed (km/h)") +
  theme_minimal()

# 3. Correlation heatmap of all numeric variables
num_vehicle <- vehicle_df[, c("Engine_Size", "Horsepower", "Fuel_Efficiency", "Top_Speed", "Safety_Rating")]
corr_matrix <- cor(num_vehicle)
corrplot(corr_matrix, method = "color", addCoef.col = "black",
  title = "Correlation Heatmap - Vehicle Performance", mar = c(0,0,2,0))

# 4. NOTE: Tableau dashboard (bar charts + scatter plots, Engine_Size /
#   Safety_Rating filters) -> done in Tableau, not R.
```

Output Visualization

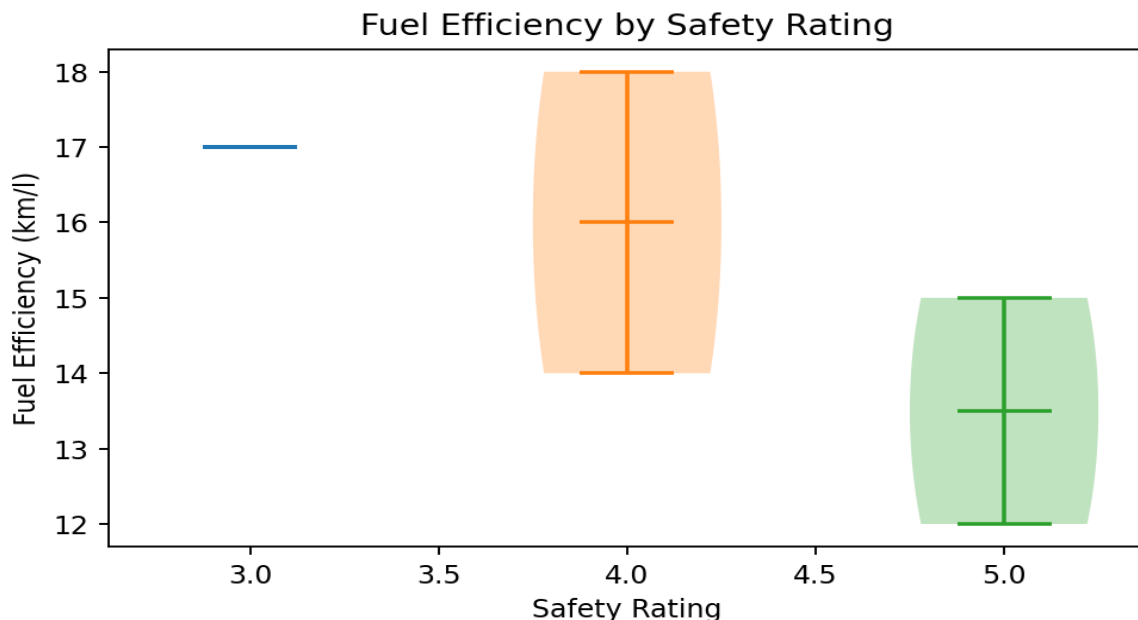


Fig 28.1 – Fuel Efficiency by Safety Rating (Violin Plot)

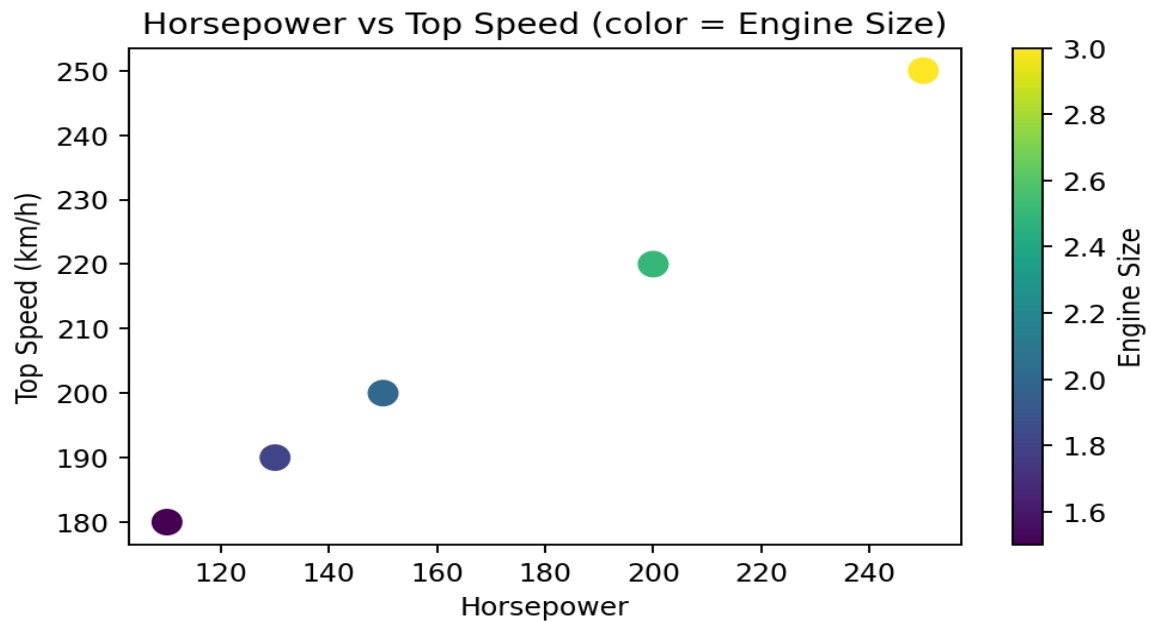


Fig 28.2 – Horsepower vs Top Speed (Color = Engine Size)

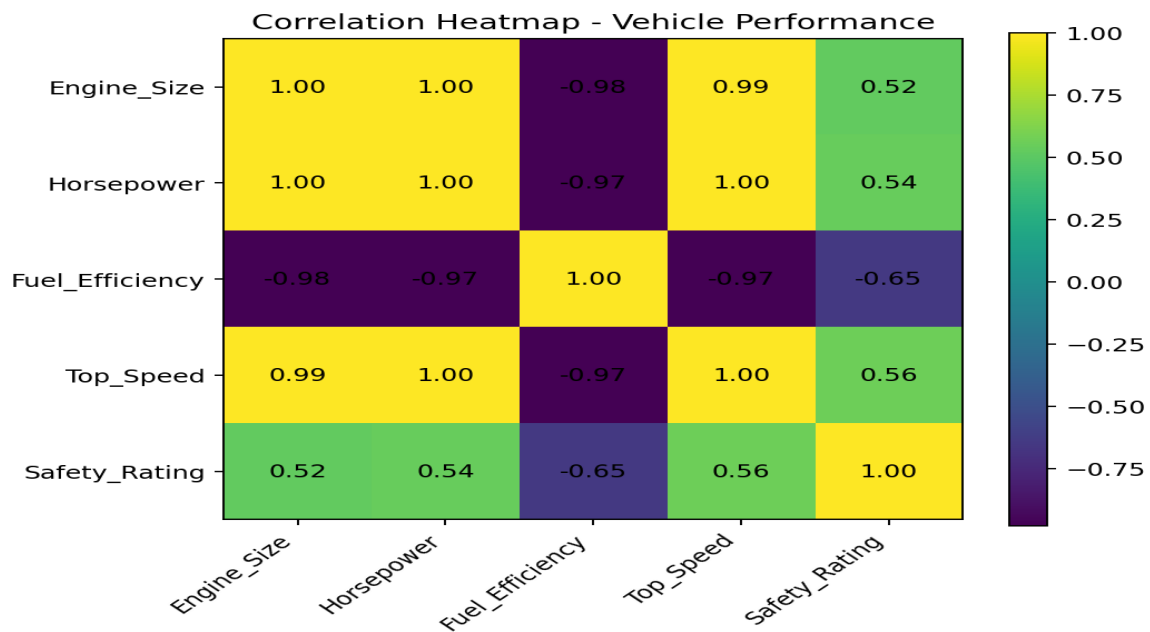


Fig 28.3 – Correlation Heatmap of Vehicle Performance Variables

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

vehicle_df = pd.DataFrame({
    "Vehicle_ID": ["V1", "V2", "V3", "V4", "V5"],
    "Engine_Size": [1.5, 2.0, 3.0, 2.5, 1.8],
    "Horsepower": [110, 150, 250, 200, 130],
    "Fuel_Efficiency": [18, 15, 12, 14, 17],
    "Top_Speed": [180, 200, 250, 220, 190],
    "Safety_Rating": [4, 5, 5, 4, 3]
})

# 1. Violin plot: Fuel_Efficiency across Safety_Rating categories
sns.violinplot(data=vehicle_df, x="Safety_Rating", y="Fuel_Efficiency",
               hue="Safety_Rating", legend=False)
plt.title("Fuel Efficiency by Safety Rating")
plt.show()

# 2. Scatter: Horsepower vs Top_Speed, color by Engine_Size
sc = plt.scatter(vehicle_df["Horsepower"], vehicle_df["Top_Speed"],
                 c=vehicle_df["Engine_Size"], cmap="coolwarm", s=100)
```

```
plt.colorbar(sc, label="Engine Size")
plt.title("Horsepower vs Top Speed (color = Engine Size)")
plt.show()

# 3. Correlation heatmap of all numeric variables
num_vehicle = vehicle_df[["Engine_Size", "Horsepower", "Fuel_Efficiency", "Top_Speed", "Safety_Rating"]]
corr_matrix = num_vehicle.corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap - Vehicle Performance")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.
```


SET 29: Student Academic Performance

Dataset: Student_ID, Age, Study_Hours, Attendance, Test_Score, Participation_Score (5 students)

1. Create a stacked area chart of Test_Score and Participation_Score across students.
2. Create a boxplot of Study_Hours grouped by Attendance quartiles.
3. Create a density plot of Test_Score.
4. Import into Tableau and build an interactive scatterplot: Study_Hours vs Test_Score, coloured by Participation_Score, with a filter by Age.

R Code

```
student2_df <- data.frame(
  Student_ID = c("S1", "S2", "S3", "S4", "S5"),
  Age = c(19, 21, 20, 22, 23),
  Study_Hours = c(12, 8, 15, 10, 7),
  Attendance = c(90, 70, 95, 85, 60),
  Test_Score = c(85, 70, 92, 80, 65),
  Participation_Score = c(8, 7, 9, 8, 6)
)

# 1. Stacked area chart: Test_Score and Participation_Score across students
student2_long <- melt(student2_df, id.vars = "Student_ID",
  measure.vars = c("Test_Score", "Participation_Score"),
  variable.name = "Metric", value.name = "Score")
ggplot(student2_long, aes(x = Student_ID, y = Score, fill = Metric, group = Metric)) +
  geom_area(position = "stack") +
  labs(title = "Test Score and Participation Score by Student", x = "Student", y = "Score") +
  theme_minimal()

# 2. Boxplot of Study_Hours grouped by Attendance quartiles
student2_df$Attendance_Quartile <- cut(student2_df$Attendance,
  breaks = quantile(student2_df$Attendance, probs = seq(0,1,0.25)),
  include.lowest = TRUE,
  labels = c("Q1", "Q2", "Q3", "Q4"))
ggplot(student2_df, aes(x = Attendance_Quartile, y = Study_Hours, fill = Attendance_Quartile)) +
  geom_boxplot() +
  labs(title = "Study Hours by Attendance Quartile", x = "Attendance Quartile", y = "Study Hours") +
  theme_minimal()

# 3. Density plot of Test_Score
ggplot(student2_df, aes(x = Test_Score)) +
  geom_density(fill = "lightblue", alpha = 0.6) +
  labs(title = "Density Plot of Test Scores", x = "Test Score", y = "Density") +
  theme_minimal()

# 4. NOTE: Tableau interactive scatterplot (Study_Hours vs Test_Score,
# color by Participation_Score, filter by Age) -> done in Tableau.
```

Output Visualization

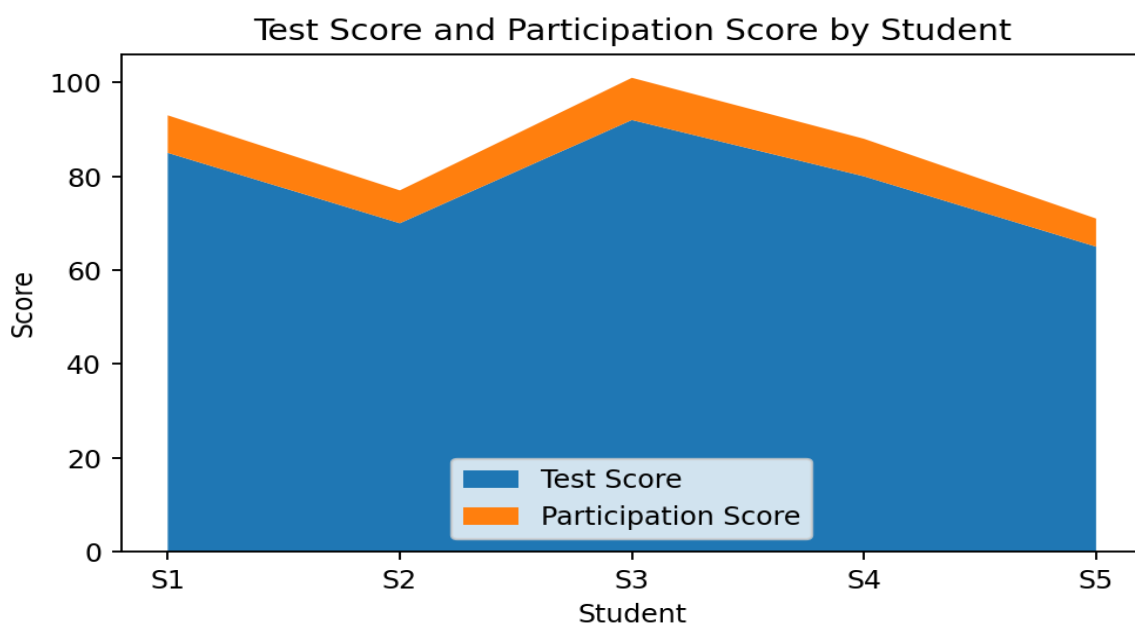


Fig 29.1 – Test Score and Participation Score by Student (Stacked Area)

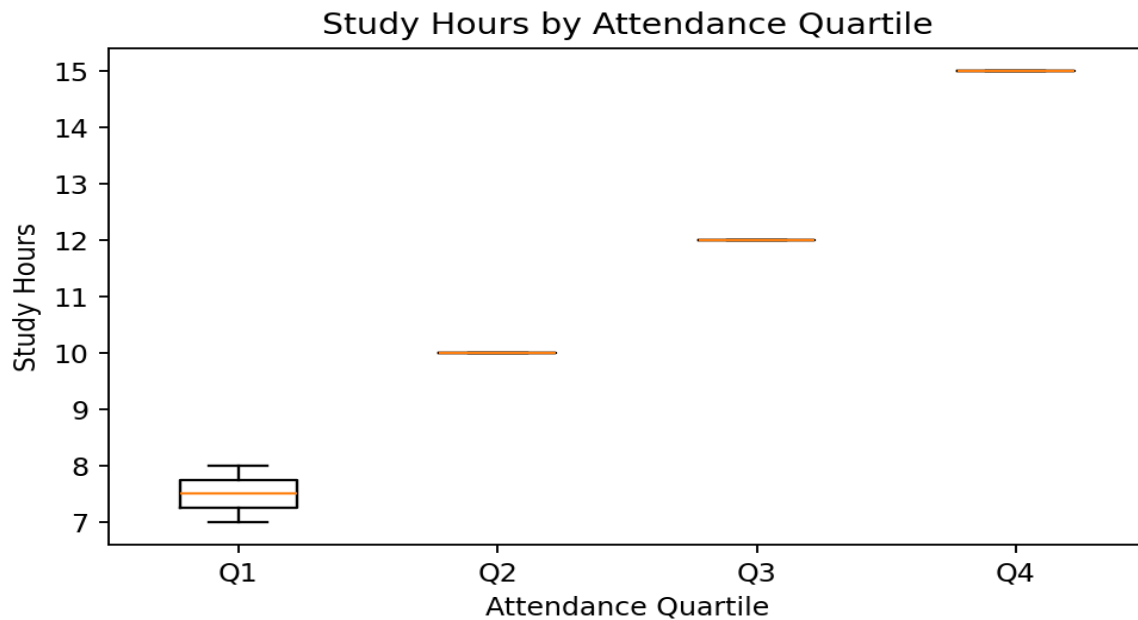


Fig 29.2 – Study Hours by Attendance Quartile (Boxplot)

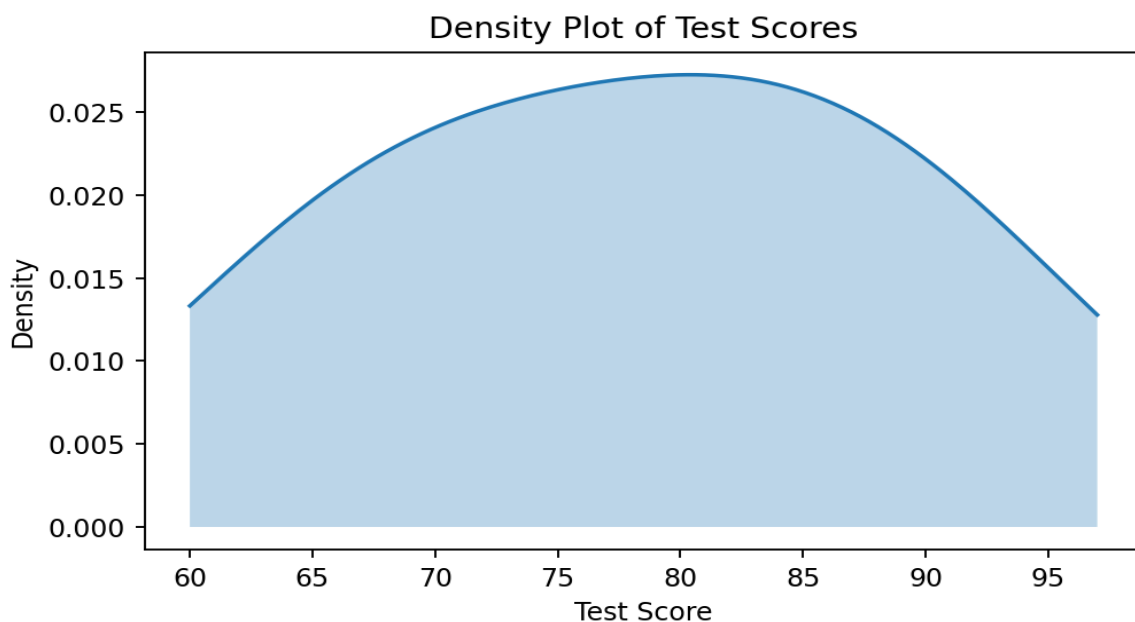


Fig 29.3 – Density Plot of Test Scores

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

student2_df = pd.DataFrame({
    "Student_ID": ["S1", "S2", "S3", "S4", "S5"],
    "Age": [19, 21, 20, 22, 23],
    "Study_Hours": [12, 8, 15, 10, 7],
    "Attendance": [90, 70, 95, 85, 60],
    "Test_Score": [85, 70, 92, 80, 65],
    "Participation_Score": [8, 7, 9, 8, 6]
})

# 1. Stacked area chart: Test_Score and Participation_Score across students
plt.stackplot(student2_df["Student_ID"],
              student2_df["Test_Score"], student2_df["Participation_Score"],
              labels=["Test_Score", "Participation_Score"])
plt.title("Test Score and Participation Score by Student")
plt.legend()
plt.show()

# 2. Boxplot of Study_Hours grouped by Attendance quartiles
```

```

student2_df["Attendance_Quartile"] = pd.qcut(student2_df["Attendance"], 4,
                                             labels=["Q1", "Q2", "Q3", "Q4"])
sns.boxplot(data=student2_df, x="Attendance_Quartile", y="Study_Hours",
            hue="Attendance_Quartile", legend=False)
plt.title("Study Hours by Attendance Quartile")
plt.show()

# 3. Density plot of Test_Score
sns.kdeplot(student2_df["Test_Score"], fill=True, color="lightblue")
plt.title("Density Plot of Test Scores")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.

```

SET 30: Mobile App Usage

Dataset: User_ID, Gender, Age, Screen_Time, App_Usage_Count, Data_Used, Satisfaction, Usage_Date (6 users)

1. Create a histogram with an overlaid density plot for Screen_Time.
2. Create a scatter plot of Data_Used vs Screen_Time with a correlation coefficient and trend line.
3. Compute the average Satisfaction by Gender and create a bar chart with value labels.
4. Import into Tableau and build a bubble chart (Screen Time vs Data Used, coloured by Gender, sized by App Usage Count), a line chart for usage trends, with filters for Gender/Usage_Date, in a mini dashboard.

R Code

```
app_df <- data.frame(
  User_ID = c("U01", "U02", "U03", "U04", "U05", "U06"),
  Gender = c("Male", "Female", "Male", "Female", "Male", "Female"),
  Age = c(20, 22, 19, 21, 23, 20),
  Screen_Time = c(4.5, 6.0, 3.2, 7.1, 2.8, 5.4),
  App_Usage_Count = c(18, 25, 12, 30, 10, 22),
  Data_Used = c(2.4, 3.8, 1.6, 4.5, 1.2, 3.1),
  Satisfaction = c(3, 5, 3, 5, 2, 4),
  Usage_Date = as.Date(c("2025-01-08", "2025-01-08", "2025-02-11",
    "2025-02-11", "2025-03-14", "2025-03-14"))
)

# 1. Histogram + density plot for Screen_Time
ggplot(app_df, aes(x = Screen_Time)) +
  geom_histogram(aes(y = ..density..), binwidth = 1, fill = "plum", color = "black") +
  geom_density(color = "darkred", size = 1) +
  labs(title = "Screen Time: Histogram + Density", x = "Screen Time (hrs)", y = "Density") +
  theme_minimal()

# 2. Scatter: Data_Used vs Screen_Time with correlation + trend line
correlation_value <- cor(app_df$Data_Used, app_df$Screen_Time)
ggplot(app_df, aes(x = Screen_Time, y = Data_Used)) +
  geom_point(size = 3, color = "steelblue") +
  geom_smooth(method = "lm", se = FALSE, color = "black") +
  labs(title = paste0("Data Used vs Screen Time (r = ", round(correlation_value, 2), ")"),
    x = "Screen Time (hrs)", y = "Data Used (GB)") +
  theme_minimal()

# 3. Average Satisfaction by Gender + bar chart with value labels
satisfaction_avg <- app_df %>% group_by(Gender) %>% summarise(AvgSatisfaction = mean(Satisfaction))
ggplot(satisfaction_avg, aes(x = Gender, y = AvgSatisfaction, fill = Gender)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = round(AvgSatisfaction, 2)), vjust = -0.5) +
  labs(title = "Average Satisfaction by Gender", x = "Gender", y = "Average Satisfaction") +
  theme_minimal() + theme(legend.position = "none")

# 4. NOTE: Tableau (bubble chart Screen Time vs Data Used, color = Gender,
# size = App Usage Count; line chart of usage trends; filters) -> Tableau.
```

Output Visualization

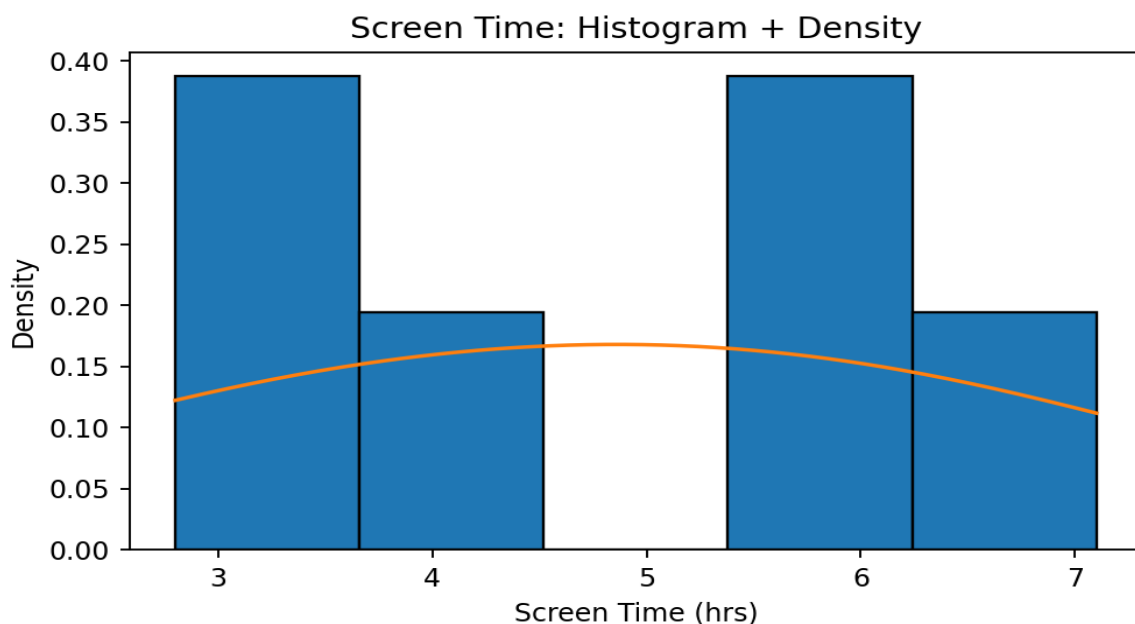


Fig 30.1 – Screen Time: Histogram + Density

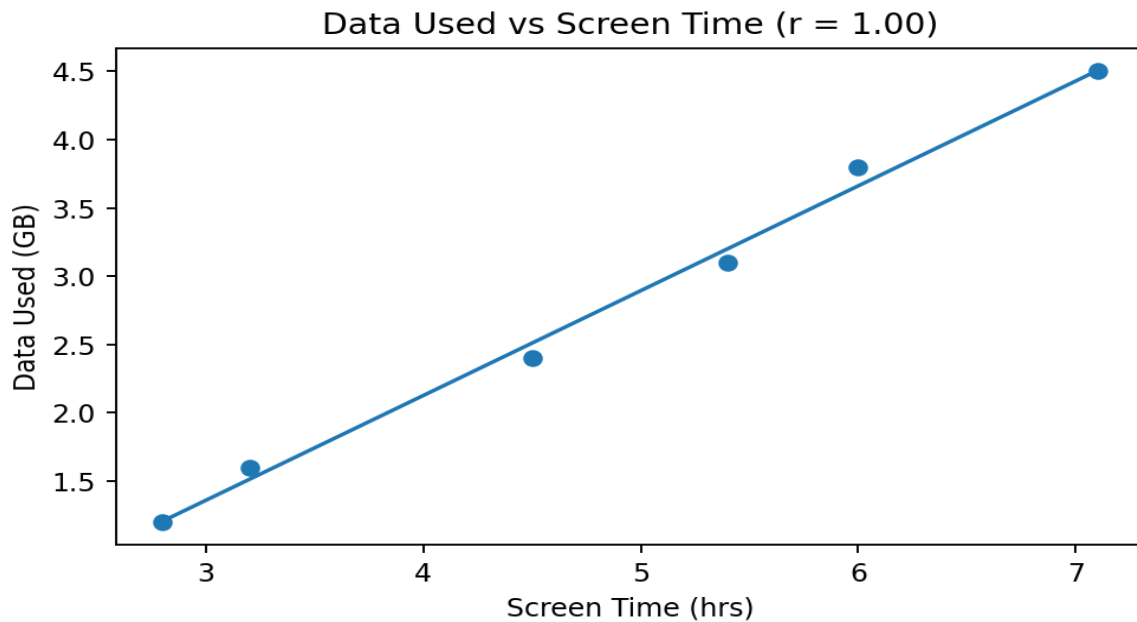


Fig 30.2 – Data Used vs Screen Time (with r-value)

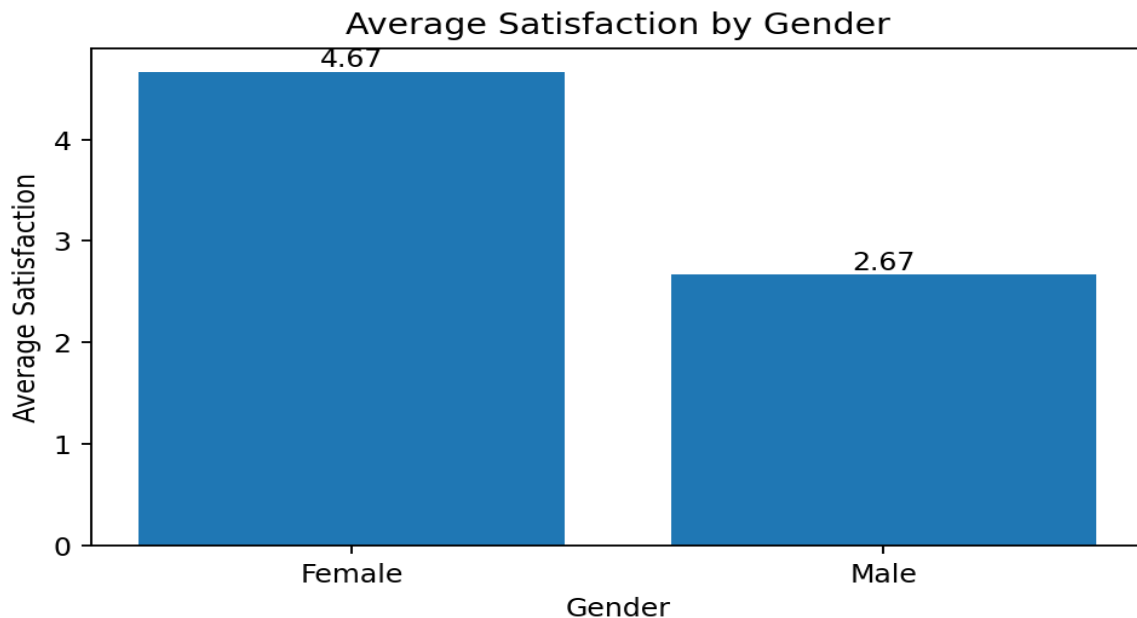


Fig 30.3 – Average Satisfaction by Gender

Note: Tableau dashboard tasks are built in Tableau Desktop, not in R/Python.

Python Code (Equivalent)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

app_df = pd.DataFrame({
    "User_ID": ["U01", "U02", "U03", "U04", "U05", "U06"],
    "Gender": ["Male", "Female", "Male", "Female", "Male", "Female"],
    "Age": [20, 22, 19, 21, 23, 20],
    "Screen_Time": [4.5, 6.0, 3.2, 7.1, 2.8, 5.4],
    "App_Usage_Count": [18, 25, 12, 30, 10, 22],
    "Data_Used": [2.4, 3.8, 1.6, 4.5, 1.2, 3.1],
    "Satisfaction": [3, 5, 3, 5, 2, 4],
    "Usage_Date": pd.to_datetime(["2025-01-08", "2025-01-08", "2025-02-11",
                                   "2025-02-11", "2025-03-14", "2025-03-14"])
})

# 1. Histogram + density plot for Screen_Time
sns.histplot(app_df["Screen_Time"], bins=5, stat="density", color="plum", edgecolor="black")
sns.kdeplot(app_df["Screen_Time"], color="darkred")
plt.title("Screen Time: Histogram + Density")
plt.show()

# 2. Scatter: Data_Used vs Screen_Time with correlation + trend line
```

```

correlation_value = app_df["Data_Used"].corr(app_df["Screen_Time"])
sns.regplot(data=app_df, x="Screen_Time", y="Data_Used", color="steelblue",
            line_kws={"color": "black"}, ci=None)
plt.title(f"Data Used vs Screen Time (r = {correlation_value:.2f})")
plt.show()

# 3. Average Satisfaction by Gender + bar chart with value labels
satisfaction_avg = app_df.groupby("Gender")["Satisfaction"].mean().reset_index()
ax = sns.barplot(data=satisfaction_avg, x="Gender", y="Satisfaction",
                hue="Gender", legend=False)
for i, v in enumerate(satisfaction_avg["Satisfaction"]):
    ax.text(i, v + 0.05, f"{v:.2f}", ha="center")
plt.title("Average Satisfaction by Gender")
plt.show()

# 4. Tableau dashboard step done in Tableau, not Python.

```